

Second-Order Methods

Newton, Gauss-Newton, and Levenberg–Marquardt

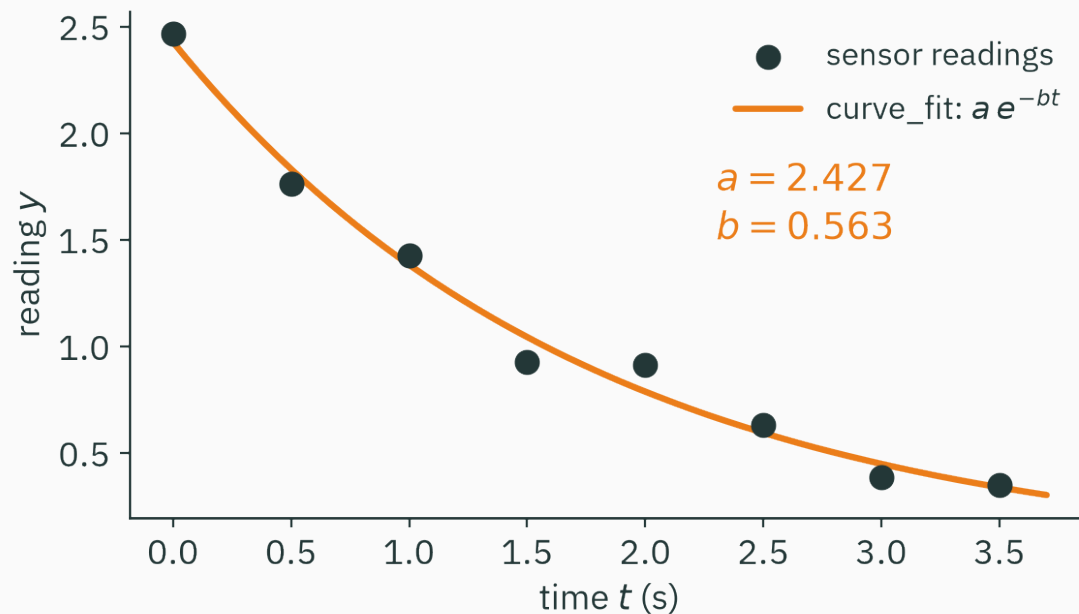
Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

Nonlinear least squares in scipy

A sensor, a decay, eight points

You're calibrating a sensor. After a pulse, its reading decays like $a e^{-bt}$ — and the datasheet needs the decay rate b . You log eight noisy readings:



Two unknowns (a, b) , eight equations that do not agree exactly. This is a nonlinear least-squares problem.

Everyone calls the same one-liner

```
from scipy.optimize import curve_fit

def model(t, a, b):
    return a * np.exp(-b * t)

popt, pcov = curve_fit(model, t, y, p0=(1.0, 0.2))
# popt = [2.4274, 0.5628]
```

No learning rate is supplied by the user; the solver adapts its step internally and returns the fitted parameters.

With no bounds supplied, `curve_fit` defaults to `method='lm'`. We will derive the core damped Gauss-Newton idea and then distinguish it from MINPACK's full implementation.

From a linear model to a quadratic model

L17's gradient descent trusted a **line** — and paid for it in steps, with the condition number κ setting the price.

When the Hessian is positive definite, the quadratic local model has a unique minimizer. Solving for it gives the Newton step.

- **Newton**: model the loss with L9's quadratic Taylor approximation and solve for the model's minimum.
- **Gauss-Newton**: when the loss is a **sum of squares**, get the curvature almost free.
- **Levenberg-Marquardt**: damp the Gauss-Newton system when its local model is unreliable.

Learning outcomes

By the end of this lecture you will be able to:

1. Derive ★ the Newton step $\delta = -H^{-1}\nabla f$ by minimizing the quadratic model.
2. Explain why exact Newton finishes a positive-definite quadratic in one step in exact arithmetic.
3. Run Newton by hand in 1-D and recognize **quadratic convergence** (digits double).
4. Price a Newton step (d^2 memory, d^3 solve) and name its failure modes (saddles, ascent).
5. Set up **nonlinear least squares**: residual vector r , its Jacobian J , $\nabla \mathcal{L} = J^\top r$.
6. Derive the **Gauss-Newton** system $(J^\top J)\delta = -J^\top r$ and interpret LM's damping parameter λ .
7. Say precisely what `scipy.optimize.curve_fit` runs.

A parabola instead of a line

L17 in one slide: the line and its price

Gradient descent came from trusting the **first-order** Taylor model for one small step:

$$f(\mathbf{x} + \boldsymbol{\delta}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^\top \boldsymbol{\delta} \quad \Rightarrow \quad \boldsymbol{\delta} = -\eta \nabla f(\mathbf{x})$$

- The learning rate η = how far you trust a line that is only true at a point.
- On a bowl with condition number κ : the safe η is set by the **steepest** wall, so the **flattest** direction crawls — the step budget grows like κ . That was the main result from L17.

A line has no bottom. However far you trust it, it only ever says “downhill is that way” — never “stop **here**”.

A positive-definite quadratic model has a minimizer

L9 built the **second-order** Taylor model and promised it would run Module 4:

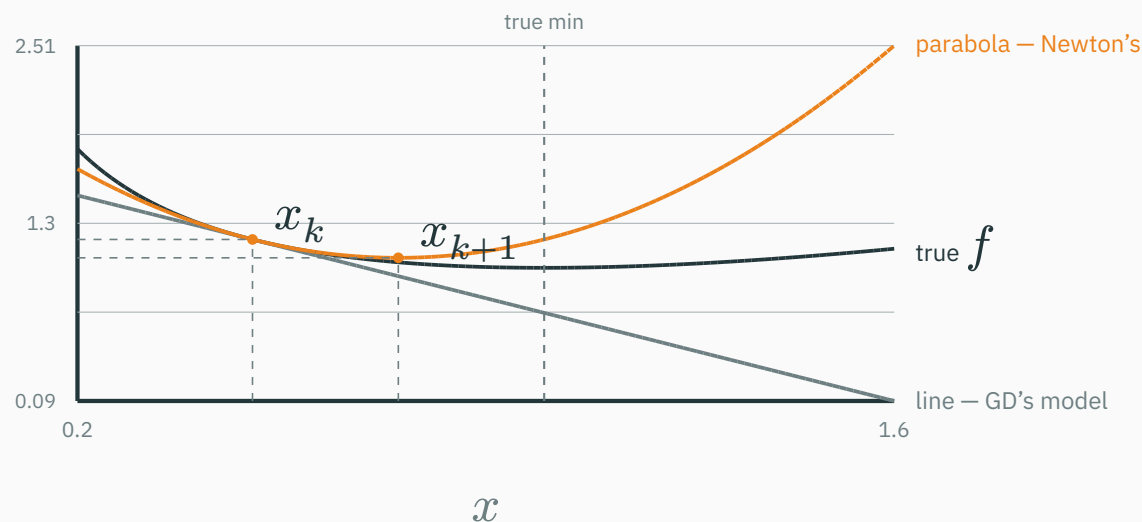
$$f(\mathbf{x} + \boldsymbol{\delta}) \approx \underbrace{f(\mathbf{x})}_{\text{height}} + \underbrace{\nabla f(\mathbf{x})^\top \boldsymbol{\delta}}_{\text{tilt}} + \underbrace{\frac{1}{2} \boldsymbol{\delta}^\top H(\mathbf{x}) \boldsymbol{\delta}}_{\text{bend}}$$

- GD kept height + tilt. Today we keep the **bend** too.
- If H is positive definite (L9: all eigenvalues > 0), this model is a **bowl** — and a bowl has a **bottom you can name**.

When the quadratic model is convex, solve directly for its minimizer.

Picture first: two models, two policies

Same function, same point, both local stories drawn at $x_k = 0.5$ (here $f(x) = x - \ln x$):



the line never bottoms out — **you** must choose a step; the parabola has a vertex — **it** chooses the step for you

The two policies, side by side

	gradient descent (L17)	Newton (today)
local model	line: $f + \nabla f^\top \delta$	bowl: $f + \nabla f^\top \delta + \frac{1}{2} \delta^\top H \delta$
the step	$-\eta \nabla f$ — direction from the model, length from you	solve for the model's minimum — no η supplied
uses	slopes only	slopes and curvature

One question remains: **where exactly is the bowl's bottom?** That formula is today's ★.

The Newton step

★ Step 1 — in 1-D: minimize the parabola

The model around x_k , as a function of the step δ :

$$q(\delta) = f(x_k) + f'(x_k) \delta + \frac{1}{2} f''(x_k) \delta^2$$

A parabola in δ — minimize it like any 1-D function (set $\frac{dq}{d\delta} = 0$, L7):

$$f'(x_k) + f''(x_k) \delta = 0 \quad \Rightarrow \quad \delta^* = -\frac{f'(x_k)}{f''(x_k)}$$

Numbers — $f(x) = x - \ln x$ at $x_k = 0.5$: $f' = 1 - \frac{1}{0.5} = -1$, $f'' = \frac{1}{0.5^2} = 4$:

$$\delta^* = -\frac{-1}{4} = 0.25 \quad \Rightarrow \quad x_{k+1} = 0.75 \quad \text{— the vertex in the picture, exactly}$$

★ Step 2 — in n -D: minimize the quadratic model

D DERIVATION

$$q(\delta) = f(x_k) + \nabla f^\top \delta + \frac{1}{2} \delta^\top H \delta$$

Differentiate with respect to δ using L9's two gradient rules ($\nabla(g^\top \delta) = g$, and the ★ rule $\nabla(\frac{1}{2} \delta^\top H \delta) = H\delta$ for symmetric H):

$$\nabla_\delta q = \nabla f + H\delta = 0$$

The Newton system: solve $H\delta = -\nabla f$. If H is invertible, this is $\delta_{\text{Newton}} = -H^{-1}\nabla f$.

This stationary point is the model's **minimum** only when H is positive definite. Indefinite or singular Hessians require safeguards.

Read the step like an engineer

$$\underbrace{\delta = -\eta \nabla f}_{\text{GD}} \quad \text{vs} \quad \underbrace{\delta = -H^{-1} \nabla f}_{\text{Newton}}$$

- When H is positive definite, Newton can be read as a gradient step preconditioned by H^{-1} .
- In the eigenbasis of a positive-definite H , direction i is scaled by $1/\lambda_i$: steep directions receive smaller components and flat directions larger ones.
- In practice you never form H^{-1} : **solve** $H\delta = -\nabla f$ (Module 1 discipline — solving is cheaper and more stable than inverting).

Numbers: one Newton step on L9's bowl

$f(x, y) = x^2 + xy + y^2$ at $(1, 1)$ — the running function of L9:

$$\nabla f = \begin{pmatrix} 2x + y \\ x + 2y \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \quad H = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \text{ (everywhere)}$$

Solve $H\delta = -\nabla f$ by hand (2×2 elimination):

$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \delta = \begin{pmatrix} -3 \\ -3 \end{pmatrix} \Rightarrow \delta = \begin{pmatrix} -1 \\ -1 \end{pmatrix} \Rightarrow x_1 = (0, 0)$$

And $\nabla f(0, 0) = \mathbf{0}$ — the **exact minimum, in one step**. (Chalkdust's solver agrees, live: $\delta = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$.) Not a coincidence — the next slide shows it holds for every quadratic.

One Newton step solves any quadratic

Take **any** quadratic $f(x) = \frac{1}{2}x^\top Ax + b^\top x + c$ with A positive definite. Then (L9 rules):

$$\nabla f = Ax + b, \quad H = A \quad \text{— the same matrix at every point}$$

So from **any** starting point x_0 :

$$x_1 = x_0 - A^{-1}(Ax_0 + b) = x_0 - x_0 - A^{-1}b = -A^{-1}b$$

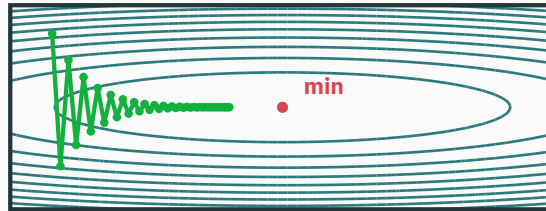
— which is exactly where $\nabla f = 0$: **the** minimum.

For a positive-definite quadratic, exact Newton reaches the minimizer in one step in exact arithmetic. Conditioning still affects the numerical stability of the linear solve.

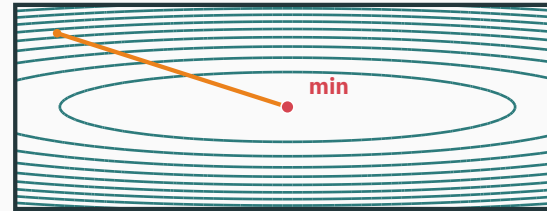
Compare gradient descent and Newton on $\kappa = 50$

Use the same start on $f = \frac{1}{2}(x^2 + 50y^2)$; both trajectories are computed in the slide:

gradient descent · 40 steps



Newton · 1 step



GD: zigzags die out, then a long crawl — 40 steps and still $\sim 23\%$ of the x -gap left. Newton: $H\delta = -\nabla f$, solved once.

For an exactly represented positive-definite quadratic, Newton still needs one iteration as κ grows. In floating-point arithmetic, an ill-conditioned Newton system can lose accuracy.

Code: Newton in five lines

```
def newton(grad, hess, x, steps=10):  
    for _ in range(steps):  
        x = x - np.linalg.solve(hess(x), grad(x))    # solve, never invert  
    return x  
  
A = np.array([[1., 0.], [0., 50.]])                # the kappa = 50 bowl  
newton(lambda x: A @ x, lambda x: A, np.array([-2.2, 0.65]), steps=1)  
# array([0., 0.])                                <- one step, exact
```

Compare L17: no `lr=`, no schedule, no divergence watch. The price appears in Section 5 — first, run it on functions that **aren't** parabolas.

Newton's method is applied to $f(x) = 3(x - 2)^2$ starting at $x_0 = 10$. Where is x_1 ?

A. $x_1 = 2$

B. $x_1 = 6$ — halfway

C. $x_1 = 4$

D. Cannot say — depends on the learning rate

Correct: A — $x_1 = 2$, the exact minimum

Why: f is quadratic, so the second-order model equals f : $f'(10) = 6(10 - 2) = 48$, $f'' = 6$, $\delta = -\frac{48}{6} = -8$, and $x_1 = 2$. Newton has no learning rate; the curvature f'' scales the gradient.

Newton beyond quadratics

For a non-quadratic objective, rebuild the model

For a non-quadratic objective, the model is local, so recompute it after every step:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - H(\mathbf{x}_k)^{-1} \nabla f(\mathbf{x}_k) \quad (1\text{-D: } x_{k+1} = x_k - f'(x_k)/f''(x_k))$$

Each step reaches the stationary point of the current model, not necessarily the function's minimum; then the model is rebuilt.

Our 1-D running example $f(x) = x - \ln x$ (min at $x^* = 1$): the update simplifies beautifully —

$$x_{k+1} = x_k - \frac{1 - 1/x_k}{1/x_k^2} = x_k - (x_k^2 - x_k) = 2x_k - x_k^2$$

No calculus left at run time. Let's run it.

You've met Newton before

Newton's method was born for **root finding** — solve $g(x) = 0$ by following the tangent:

$$x_{k+1} = x_k - \frac{g(x_k)}{g'(x_k)} \quad (\text{Newton-Raphson, the school version: } \sqrt{2} \text{ via } x_{k+1} = (x_k + 2/x_k)/2)$$

Minimizing f means solving $\nabla f = 0$. Substitute $g = f'$:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)} \quad \text{— the same formula we just derived from the parabola}$$

Two equivalent readings: solve for the parabola's vertex, or apply Newton root-finding to the gradient. A zero of ∇f need not be a minimum.

Watch the digits double

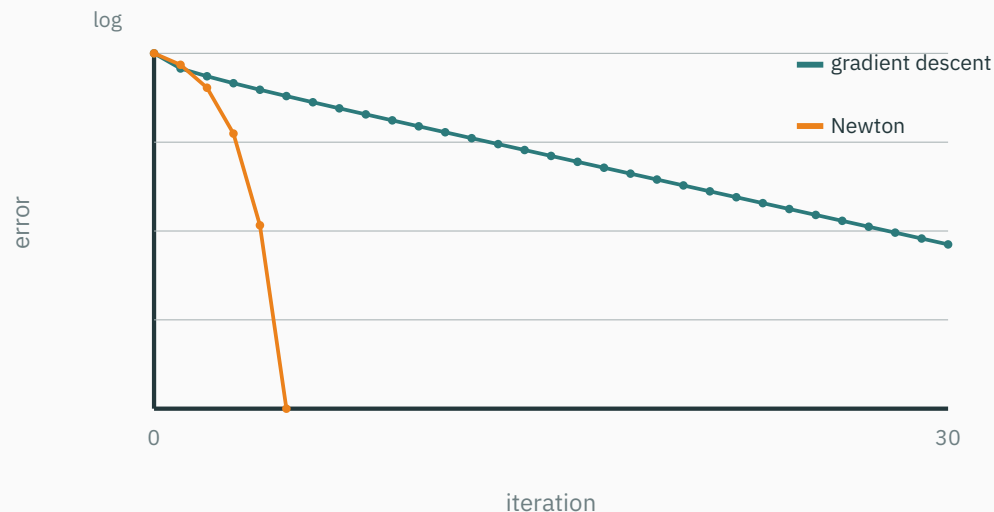
$x_{k+1} = 2x_k - x_k^2$ from $x_0 = 0.5$, run live in the slide (target $x^* = 1$):

k	x_k	error $x_k - 1$	correct digits ($-\log_{10}$)
0	0.5000000000	5.0×10^{-1}	0.3
1	0.7500000000	2.5×10^{-1}	0.6
2	0.9375000000	6.3×10^{-2}	1.2
3	0.9960937500	3.9×10^{-3}	2.4
4	0.9999847412	1.5×10^{-5}	4.8
5	0.9999999998	2.3×10^{-10}	9.6

The correct-digit count $0.3 \rightarrow 0.6 \rightarrow 1.2 \rightarrow 2.4 \rightarrow 4.8 \rightarrow 9.6$ — **doubling every step**.

Here it's exact: $x_{k+1} - 1 = 2x_k - x_k^2 - 1 = -(x_k - 1)^2$, so $e_{k+1} = e_k^2$. The error **squares**.

Same function, same start: GD ($\eta = 0.3$) vs Newton, error per iteration — log scale:



GD is a **straight line** on a log scale — a constant factor (0.7) per step. Newton's slope steepens every step: the error is squaring. 9.6 correct digits by $k = 5$ vs 5.3 by $k = 30$.

Local quadratic convergence

Near a non-degenerate minimizer, if the Hessian is Lipschitz continuous and each Newton system is solved accurately:

$$\|e_{k+1}\| \leq C\|e_k\|^2 \quad \text{— "quadratic convergence": digits double per step}$$

- GD has a linear rate on a strongly convex quadratic. Newton has a local quadratic rate; the constant C and the size of the convergence neighbourhood still depend on the problem's derivatives and conditioning.
- The qualification is **near**: far from the minimum, the quadratic approximation may be inaccurate.

Newton converges rapidly when initialized near a suitable minimum. Far away, the step can increase the objective or converge to another stationary point.

Computational cost and failure modes

Computational cost of one Newton step

For d parameters, per iteration:

	needs	memory	arithmetic
gradient descent	∇f	$O(d)$	$O(d)$ once ∇f is in hand
Newton	$\nabla f, H$, a solve	$O(d^2)$	$O(d^3)$ for the solve

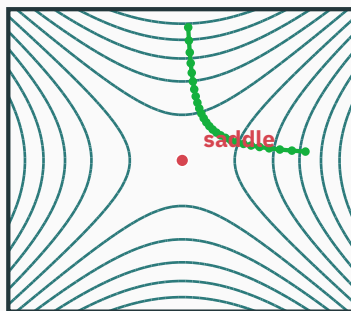
At $d = 10^6$, a dense H has 10^{12} entries $\approx$ **4 TB** in float32, before any workspace. A generic dense factorization has cubic arithmetic cost and is impractical at this scale.

Dense Newton steps can be effective at small dimension, but quadratic storage and cubic generic solves rule them out at million-parameter scale.

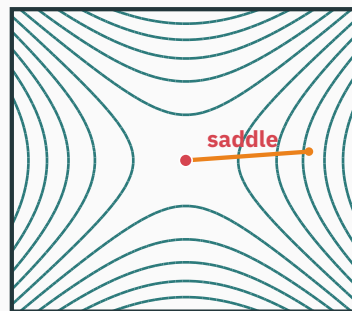
Newton can converge to a saddle point

Newton chases $\nabla f = 0$ — and (L16) a saddle **is** a $\nabla f = 0$ point. On $f = \frac{1}{2}(x^2 - y^2)$:

gradient descent · drifts off



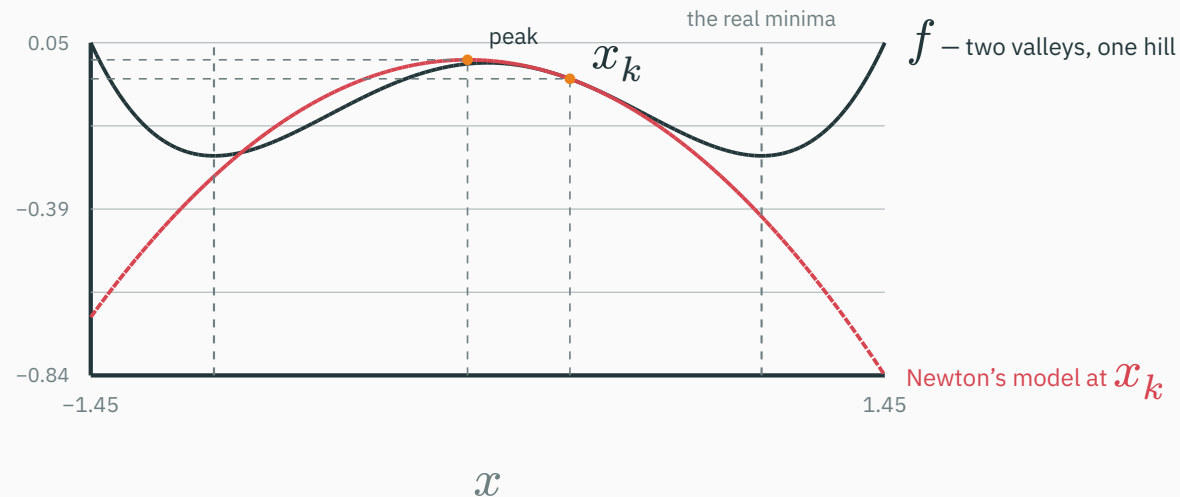
Newton · dives straight in



GD moves along the negative-curvature direction; the raw Newton step solves for the saddle's stationary point and reaches it in one step

An indefinite Hessian can produce an ascent direction

On a double well, stand at $x_k = 0.3$ where $f'' = -0.73 < 0$ — the parabola **frowns**:



a frowning parabola's stationary point is its **peak**: the “Newton step” here moves **left, toward the hilltop at $x = 0$** — and iterating converges to that local maximum

When H is not positive definite, $-H^{-1}\nabla f$ need not point downhill. Newton without a safety check is only trustworthy where the world is bowl-shaped.

Damping, line search, and trust regions

Common safeguards modify or restrict the raw Newton step:

- **Damping**: solve $(H + \tau I) \delta = -\nabla f$. Eigenvalues shift by τ ; the model becomes positive definite only when $\tau > -\lambda_{\min}(H)$ (strictly, with margin).
- **Line search**: take the Newton **direction**, but test the length before committing.
- **Trust region**: only believe the model inside a radius; grow or shrink the radius by results.

The same damping idea appears next in nonlinear least squares, with an implementation-specific trust-region rule for choosing the parameter.

Sums of squares: nonlinear least squares

Back to the sensor: the loss is a sum of squares

Our eight readings, and the model $\hat{y}(t) = a e^{-bt}$ with $\theta = (a, b)$:

t_i	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5
y_i	2.465	1.762	1.426	0.928	0.914	0.632	0.385	0.352

Per point, a **residual** — the model's miss; the loss adds the squares (L14: this is the Gaussian NLL):

$$r_i(\theta) = a e^{-bt_i} - y_i, \quad \mathcal{L}(\theta) = \frac{1}{2} \sum_{i=1}^8 r_i(\theta)^2 = \frac{1}{2} \|\mathbf{r}(\theta)\|^2$$

The $\frac{1}{2}$ is cosmetics: it cancels the 2 that differentiation drops. This **shape** — squared residuals — is everywhere: calibration, GPS, camera bundle adjustment, regression.

The residual vector and its Jacobian

$r : \mathbb{R}^2 \rightarrow \mathbb{R}^8$ — two parameters in, eight misses out. Its derivative is a **Jacobian** (L9): one row per residual, one column per parameter, 8×2 :

$$J_{i1} = \frac{\partial r_i}{\partial a} = e^{-bt_i}, \quad J_{i2} = \frac{\partial r_i}{\partial b} = -a t_i e^{-bt_i}$$

At the bad first guess $\theta_0 = (1.0, 0.2)$, three of the eight rows:

row	t_i	$\partial r_i / \partial a$	$\partial r_i / \partial b$
$i = 1$	0.0	1.000	0.000
$i = 2$	0.5	0.905	-0.452
$i = 8$	3.5	0.497	-1.738

Late-time rows barely feel a but feel b strongly — the Jacobian **is** the sensitivity table.

The gradient in terms of the Jacobian

Chain rule on $\mathcal{L} = \frac{1}{2} \mathbf{r}^\top \mathbf{r}$, shapes doing the bookkeeping (L9's habit):

$$\nabla \mathcal{L} = \underbrace{J^\top}_{2 \times 88 \times 1} \underbrace{\mathbf{r}}_{88 \times 1} \in \mathbb{R}^2$$

At $\theta_0 = (1.0, 0.2)$, with the measured residuals: $\nabla \mathcal{L} = J^\top \mathbf{r} = (-2.894, 0.937)$.

- Gradient descent is ready to go: $\theta \leftarrow \theta - \eta J^\top \mathbf{r}$. L17 machinery, nothing new.
- But Newton wants H — second derivatives of all eight r_i : expensive. **The sum-of-squares structure supplies the curvature far more cheaply** — that is Gauss-Newton, next.

**Gauss-Newton: curvature
almost for free**

The idea: linearize the residuals, not the loss

Replace each residual by its L9 linear approximation at the current θ :

$$r(\theta + \delta) \approx r + J\delta \quad \Rightarrow \quad \mathcal{L}(\theta + \delta) \approx \frac{1}{2} \|r + J\delta\|^2$$

Look at what the right side **is**: a **linear least-squares problem in δ** — the one problem Module 1 taught us to solve exactly (and L16 proved convex).

A line inside a square is a parabola: linearizing r hands us a quadratic model of $\mathcal{L}$ — without ever touching a second derivative.

The Gauss-Newton step

Minimize $\frac{1}{2}\|r + J\delta\|^2$ over δ : its gradient is $J^\top(r + J\delta) = 0$ — the **normal equations** of the linearized fit:

Gauss-Newton: solve the linear least-squares problem $\min_{\delta} \|r + J\delta\|^2$. If J has full column rank, its normal equations are $(J^\top J)\delta = -J^\top r$.

Pattern-match against Newton's $H\delta = -\nabla f$:

- right side: $J^\top r = \nabla \mathcal{L}$ ✓ (last slide) — same gradient,
- left side: $J^\top J$ **standing where the Hessian stands**. Is it the Hessian? Almost —

Relation to Newton's Hessian

Differentiate $\nabla \mathcal{L} = J^\top r$ once more (product rule — each factor depends on θ):

$$H = J^\top J + \sum_{i=1}^n r_i \nabla^2 r_i$$

Gauss-Newton keeps the first term, **drops the second**. When is that fair?

- $r_i \approx 0$ near a good fit — the dropped term is weighted by the **misses**;
- or r_i nearly linear ($\nabla^2 r_i \approx 0$) — nothing to drop.

$J^\top J$ uses first derivatives only and is always PSD. It is positive definite, and the Gauss-Newton step unique, when J has full column rank.

Numbers: the first step from $\theta_0 = (1.0, 0.2)$

Assemble the 2×2 system from the real data (all eight rows summed — check any entry by hand):

$$J^\top J = \begin{pmatrix} 4.403 & -5.488 \\ -5.488 & 11.938 \end{pmatrix}, \quad J^\top \mathbf{r} = \begin{pmatrix} -2.894 \\ 0.937 \end{pmatrix}$$

$$\begin{pmatrix} 4.403 & -5.488 \\ -5.488 & 11.938 \end{pmatrix} \boldsymbol{\delta} = \begin{pmatrix} 2.894 \\ -0.937 \end{pmatrix} \Rightarrow \boldsymbol{\delta} = \begin{pmatrix} 1.310 \\ 0.524 \end{pmatrix} \Rightarrow \boldsymbol{\theta}_1 = (2.310, 0.724)$$

From $(1.0, 0.2)$ straight to $(2.31, 0.72)$ — one solve of a 2×2 system.

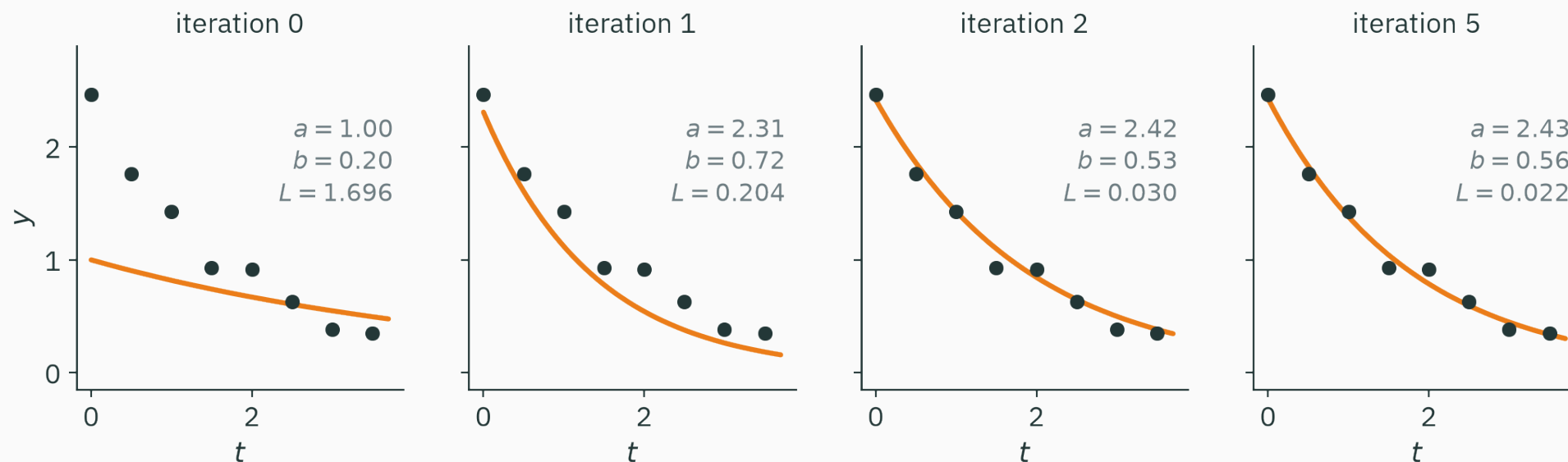
b overshoots the fitted value 0.563 on the first step because the residual linearization is local. Recompute it and continue.

The run: five solves and the loss stops changing

The whole optimization, on the real data (this table is the figure script's actual output):

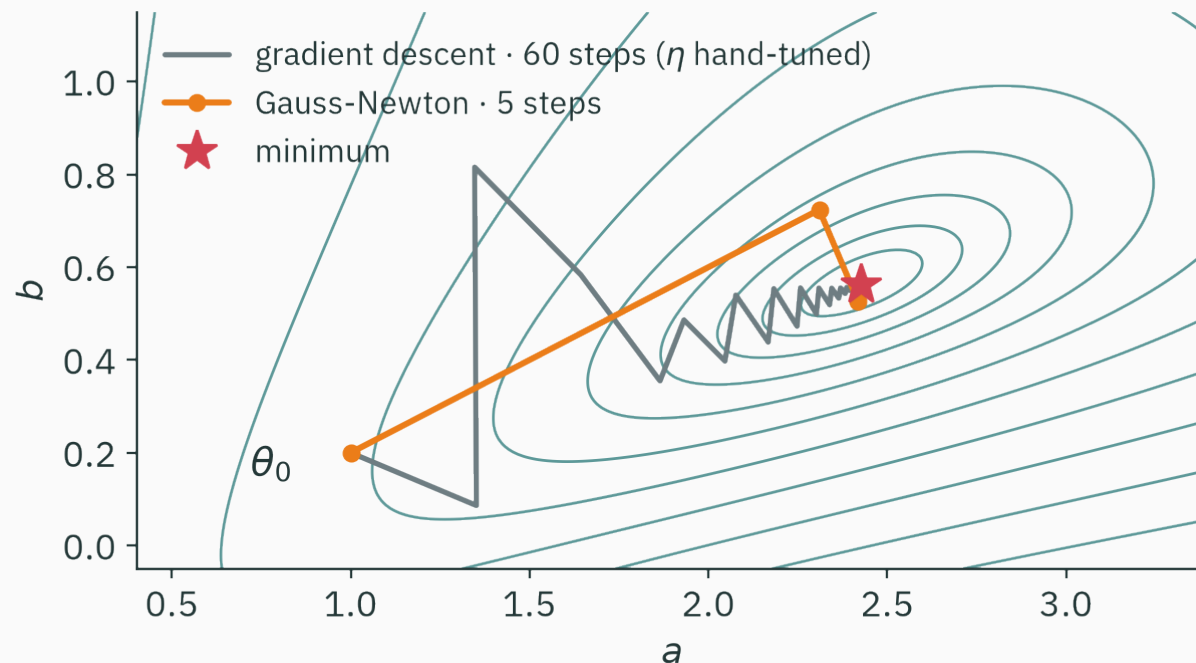
k	a_k	b_k	loss $\mathcal{L}(\theta_k)$
0	1.000000	0.200000	1.69626671
1	2.309903	0.723655	0.20392293
2	2.418643	0.525822	0.03006333
3	2.425973	0.560925	0.02159785
4	2.427402	0.562773	0.02158193
5	2.427436	0.562798	0.02158193

b overshoots and then settles as each residual linearization is recomputed. By $k = 5$ the loss has stopped changing at the displayed precision; this example alone does not establish a general quadratic rate for Gauss-Newton.



The first solve moves close to the data; by iteration 2 the remaining error is comparable to the observation noise.

Parameter trajectory in the (a, b) plane



gradient descent zigzags into the curved valley — and its η was hand-tuned **using $J^\top J$'s eigenvalues**; 25% more η and it never converges. Gauss-Newton: two visible hops, three invisible ones.

Code: Gauss-Newton in eight lines

```
theta = np.array([1.0, 0.2])           # (a, b) – the bad guess
for _ in range(6):
    a, b = theta
    r = a * np.exp(-b * t) - y         # residuals           (8,)
    J = np.stack([np.exp(-b * t),      # dr/da              (8, 2)
                  -a * t * np.exp(-b * t)], axis=1)
    delta = np.linalg.lstsq(J, -r, rcond=None)[0] # avoids forming J^T J
    theta = theta + delta
# theta = [2.427437, 0.562798]
```

Residuals, Jacobian, one small solve — no Hessian code anywhere, and no learning rate.

Levenberg–Marquardt damping

When the Gauss-Newton model is unreliable

Gauss-Newton can fail when its local residual model is inaccurate or poorly conditioned:

- Far from the fit, the linearization lies — our very first step overshoot b by 29%.
- If the data barely constrains a parameter, $J^\top J$ is **nearly singular** (L6: tiny singular values) — the solve amplifies noise into a **wild** step.
- Nothing stops a step from **increasing** the loss — the update never checks it.

Section 5 already prescribed the fix: **damp** the system — add λI before solving.

The damping parameter λ

Levenberg damping: $(J^\top J + \lambda I) \delta = -J^\top r$. Practical LM implementations often use a scaled diagonal and choose λ through a trust-region calculation.

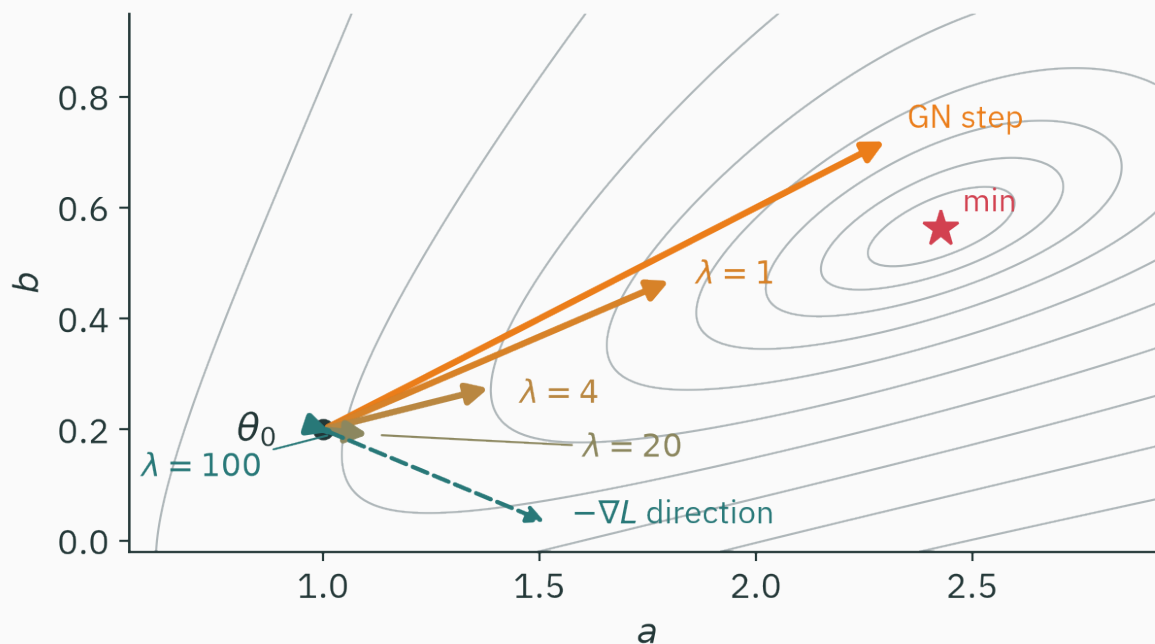
The two limiting cases explain the effect of λ :

- $\lambda \rightarrow 0$: the damping vanishes — **pure Gauss-Newton**, full quadratic confidence.
- λ large: λI dominates $J^\top J$, so $\delta \approx -\frac{1}{\lambda} J^\top r = -\frac{1}{\lambda} \nabla \mathcal{L}$ — a **small gradient-descent step**.

In this simplified system, one scalar interpolates between the Gauss-Newton step and a small negative-gradient step.

How λ changes the step

All steps solved from the same θ_0 on the real problem — only λ moves:



as λ increases, the step shrinks and rotates from the Gauss-Newton step toward the $-\nabla \mathcal{L}$ direction

Adapt λ using the observed loss

A simple classroom controller adjusts λ after evaluating a trial step:

trial step...	action	meaning
lowered the loss	accept; decrease λ	allow a step closer to Gauss-Newton
raised the loss	reject; increase λ	shrink and rotate the step toward the gradient

A real LM implementation compares predicted and observed reduction and updates its trust parameter accordingly. Its convergence rate depends on the residuals, Jacobian rank, and local regularity.

This is Section 5's trust-region idea compressed into a scalar: λ small = big trust region, λ large = tiny one.

Reproduce `curve_fit` on the sensor data

For this unconstrained problem, `curve_fit(..., method='lm')` calls SciPy's `leastsq` wrapper around MINPACK. With no `jac` supplied, MINPACK's `lmdif` estimates J by forward differences (L10) and computes a scaled trust-region Levenberg–Marquardt step using QR-based linear algebra:

```
popt, pcov = curve_fit(model, t, y, p0=(1.0, 0.2))
# popt      = [2.427437, 0.562798]      <- curve_fit (LM)
# our GN k=5: [2.427436, 0.562798]      <- this lecture, 8 lines
```

The hand-coded Gauss-Newton iteration and MINPACK reach the same fitted parameters to six decimals on this well-started example.

`curve_fit(method='lm')` uses MINPACK's modified Levenberg–Marquardt algorithm; it is more sophisticated than the multiply/divide- λ teaching rule above.

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/optimizers-beyond

Compare Newton and BFGS with gradient descent on 2-D objectives; inspect the quadratic, saddle, and poorly initialized cases, then add damping.

Try the ill-conditioned bowl first: it is this lecture's $\kappa = 50$ picture, live.

In the simplified $(J^\top J + \lambda I)$ damping system, increasing λ makes the step approach...

A. the pure Gauss-Newton step

B. a tiny step along $-\nabla \mathcal{L}$

C. the exact Newton step (with the true Hessian)

D. a step of fixed length in a random descent direction

Correct: B — a tiny step along $-\nabla \mathcal{L}$

Why: For large λ , $(J^\top J + \lambda I) \approx \lambda I$, so $\delta \approx -\frac{1}{\lambda} J^\top r = -\frac{1}{\lambda} \nabla \mathcal{L}$. The direction approaches the negative gradient and the length shrinks as $1/\lambda$. A diagonally scaled LM implementation approaches a correspondingly scaled gradient direction.

Summary & what's next

Lecture 18 — summary

- **Newton** ★: minimize the L9 quadratic model $\rightarrow$ solve $H\delta = -\nabla f$.
- **Quadratics**: one exact-arithmetic Newton step reaches the minimizer from anywhere.
- **Local rate**: nonsingular, Lipschitz Hessian $\Rightarrow$ the error squares — digits double.
- **Cost and failure modes**: d^2 memory, $O(d^3)$ solve; indefinite H needs safeguards.
- **Least squares** $\mathcal{L} = \frac{1}{2}\|\mathbf{r}\|^2$: $\nabla \mathcal{L} = J^\top \mathbf{r}$, $H = J^\top J + \sum_i r_i \nabla^2 r_i$.
- **Gauss-Newton**: drop $\sum_i r_i \nabla^2 r_i$ — $(J^\top J)\delta = -J^\top \mathbf{r}$; five solves fit our sensor.
- **Levenberg–Marquardt**: damp with an adaptive λ ; `curve_fit` runs MINPACK's LM.

Read before L19 — MML §7.1.3 · Solomon, *Numerical Algorithms*, Ch. 9 (nonlinear least squares & Gauss-Newton). **T9**: Newton and Gauss-Newton by hand and in NumPy, on this lecture's data.

Practice problems

Try on paper; verify against `np.linalg.solve` and `curve_fit`.

1. Run one Newton step on $f(x) = x^2 - 4x + 3$ from $x_0 = 0$. Why is one step enough — and what does GD with $\eta = 0.1$ leave undone after 10 steps?
2. On $f(x, y) = x^2 + xy + y^2$ from $(3, -1)$: solve $H\delta = -\nabla f$ by hand and verify you land at $(0, 0)$.
3. For $f(x) = x - \ln x$, show the Newton update is $x_{k+1} = 2x_k - x_k^2$ and that $e_{k+1} = e_k^2$ exactly. How many steps from $x_0 = 0.5$ to 300 correct digits?
4. Run Newton on the double well $f(x) = x^4/4 - x^2/2$ from $x_0 = 0.3$ for three steps. Where is it converging, and what is the sign of f'' along the way?
5. For the model $\hat{y} = a \sin(bt)$: write r_i and both columns of J , then the Gauss-Newton system at a given (a, b) .
6. In LM, show $\delta(\lambda) \rightarrow -\frac{1}{\lambda} \nabla \mathcal{L}$ as $\lambda \rightarrow \infty$. Which **two** properties of the step does λ therefore control simultaneously?



Why deep learning still runs first-order

★ OPTIONAL

If Newton is this good, why is every LLM ($d \approx 10^9$) trained with Adam-flavored GD (L17 ★★★)?

per step	GD/Adam	Newton
memory	$O(d)$	$O(d^2)$: $\sim 4 \times 10^9$ GB
arithmetic	$O(d)$ past the gradient	$O(d^3)$: 10^{27} FLOPs — years
minibatches	cheap unbiased gradients	noisy curvature estimates
indefinite curvature	may escape strict saddles	attracted to saddle points

Large-scale alternatives: **L-BFGS** (built from gradient history), Hessian-vector products Hv at near-gradient cost, structured approximations (K-FAC, Shampoo).

Newton doesn't step downhill — it teleports to the bottom of its local bowl.

Next: **Constrained Optimization: Lagrange Multipliers** — so far the search was free; real problems have fences.